Saturation

MDE - Advanced Engineering Topics, ITESO

Gomez Enriquez, Gerardo

gerardo.gomez@iteso.mx

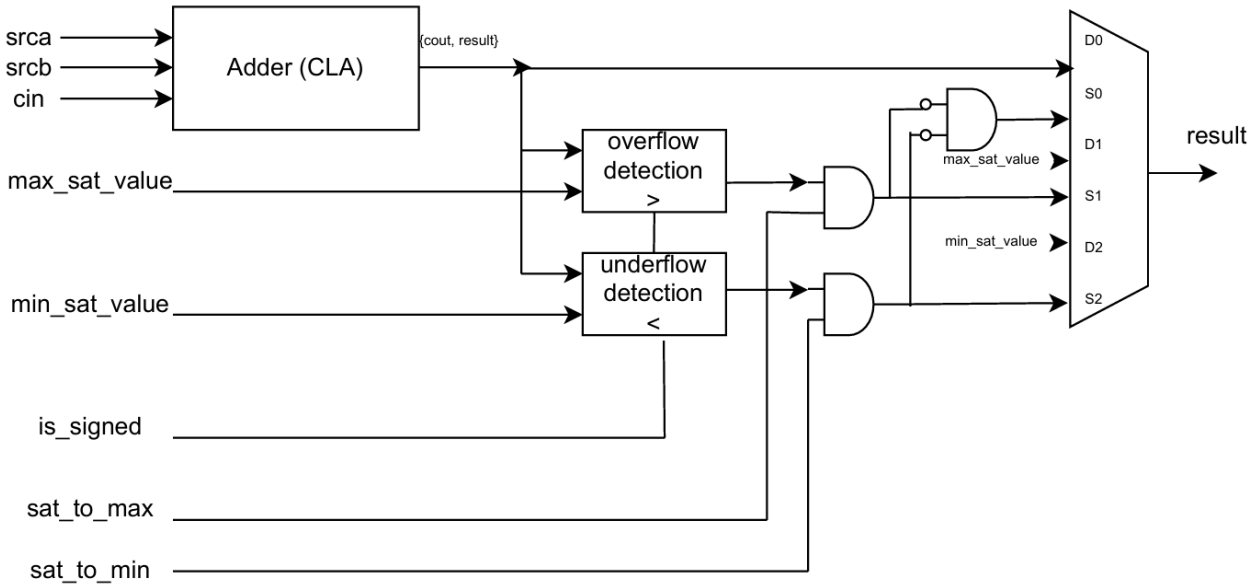
Guadalajara, Mexico

Architecture and considerations

This is an adder with saturation that can saturate to a maximum and a minimum configurable / dynamic value. Both saturations can be enabled/disabled with another set of signals.

The adder used is a Carry Look-ahead Adder (CLA) and the saturation logic is in series with the result and carry-out. The saturation logic consists of signed and unsigned comparators that check the configured saturation values with the adder result (concatenated with its carry-out for complete adder precision). The final output result is muxed depending on the overflow / underflow detection and the saturation enablement.

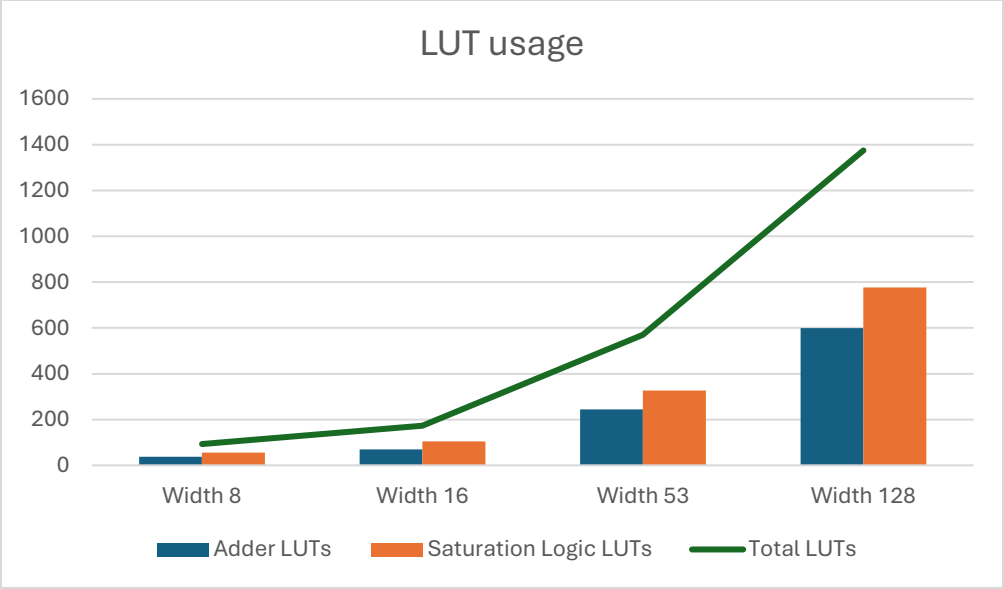
Since the saturation values are configurable / dynamic the overflow and carry-out output signals from the adder cannot be used directly with as in other adder with saturation architectures.



Area / resource usage analysis

LUT usage

Width	Adder LUTs	Saturation Logic LUTs	Total LUTs	Saturation Logic %
8	37	56	93	60.20%
16	69	104	173	60.10%
53	244	326	570	57.20%
128	599	776	1,375	56.40%

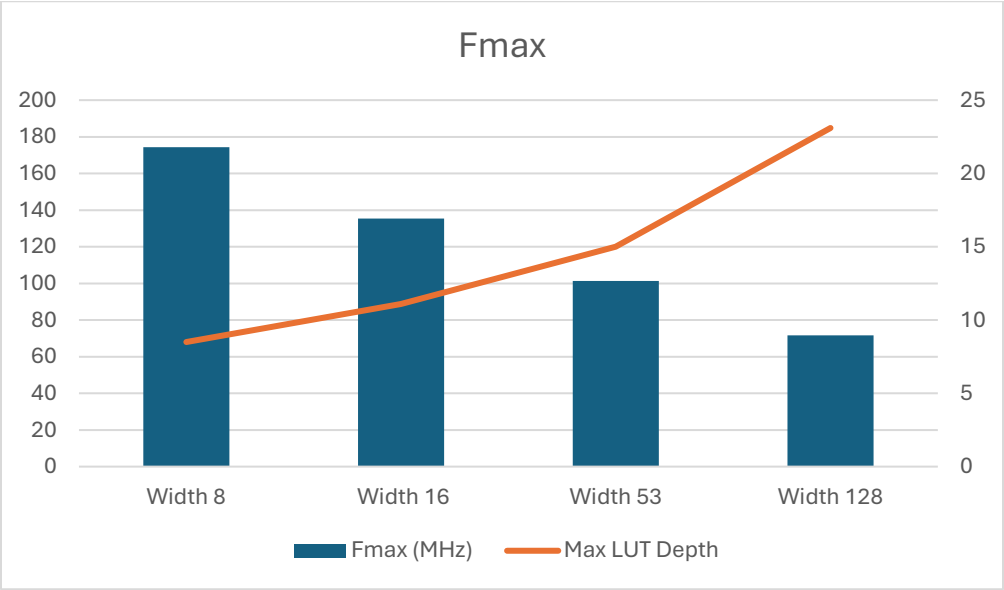


- The cost of the saturation logic in this architecture is higher than the adder (CLA) logic itself in all width configurations.
- The saturation logic consistently accounts for ~57–60% of the total LUTs, with the fraction slightly decreasing at wider widths. The CLA grows linearly with width, and its two comparators drive the saturation cost

Timing / latency analysis

Fmax & Max logic depth

Width	Fmax (MHz)	Max LUT Depth
8	174.25	8.5
16	135.34	11.1
53	101.32	15
128	71.68	23.1



- The saturation logic makes heavy use of arith cells. Quartus weighs arith-mode LUT cells (carry-chain adder cells) at 0.5 depth instead of 1.0, since they use dedicated carry hardware rather than the LUT fabric.
- Both Fmax and logic depth scale as expected with width. The fractional depths reflect the mixed arith + normal LUT path.

Critical paths

The critical path is the same for all widths: *src*_q[*] → result_q[*]*. It goes through the same logic elements, summarized below for the different width implementations.

Logic description	Node	W8 (LoL=8.5)	W16 (LoL=11.1)	W53 (LoL=15)	W128 (LoL=23.1)
CLA — LCU carry-generate/propagate	cla|gen_lvl[*].gen_lcu[*].lcu4|gg/c|combout	3	5	8	11
CLA — FA propagate	cla|gen_fa[*].fa_gp|p|combout	—	1	—	—
CLA — FA sum	cla|gen_fa[*].fa_gp|sum|combout	1	2	1	1
Comparator carry chain	LessThan*~N|cout → ~M|combout	2	2	16	27+
Saturation select	sel_max/min_sat_value~0|combout	1	1	1	1
Result MUX	result[*]~N|combout	2	2	2	1

Notice that the logic levels are weighted by Quartus differently for arithmetic more logic such as the dedicated hardware for carry chain (*LessThan*~N|cout*).

Other timing parameters

- **Latency**: 1 cycle (single cycle)
- **Throughput**: 1 operation per cycle
- **Slack**: Significantly negative. Frequency was set to 1 GHz for analysis purposes and getting the maximum possible frequency, which was 174.25 MHz when width=8.

Conclusion

The CLA contributes a roughly logarithmic number of logic levels as width grows, consistent with its look-ahead carry structure. The saturation comparator, however, grows linearly because *$signed(ext_result) > $signed(ext_max_sat_value)* synthesizes as a ripple-carry chain over WIDTH+1 bits with no look-ahead.

The saturation logic (two WIDTH+1-bit comparators plus the output MUX) is structurally heavier than the CLA in terms of area. Each comparator requires roughly as many cells as a ripple-carry adder of the same width, so two comparators alone already outweigh the CLA in LUT count.

Replacing the ripple-carry comparators with CLA-style comparators — or reusing the adder's own carry and overflow flags to detect saturation — would recover both Fmax and a substantial fraction of the LUT budget.